Fluxora Master Engineering Prompt v2

Nov 1, 2026 by @Ayush Mishra

1. How to use this prompt

Paste everything from section 4 onward into Google AI Studio as one message. Sections 1 to 3 explain the reasoning for you; sections 4 to 16, 19 and 20 are the instructions; sections 17 and 18 record open and approved decisions, and list answers the coding agent needs from you. Every feature in your v1 spec is kept. Changes are listed below, each with its reason.

Change from v1	Why
Added a hard rule: show only essential details (section 3)	You asked for it. It is now both a UX rule and a code-structure rule (encapsulation).
Gemini model is no longer hard-coded to `gemini-2.5-flash`	Google's own deprecation page and developer reports show 2.5 models being retired or restricted to existing users. Model id now comes from Firebase Remote Config.
UPI "verified success" now means both people confirm	A UPI app launch gives no proof of payment, and a personal payment has no server callback. Payer marks paid, receiver confirms received.
Manual VPA typing removed from the main payment path	NPCI guidance from February 2026 pushes mobile payments to intent flows, not typed VPAs. Saved payees and QR scan replace typing.
DPDP Act compliance added	India's data-protection rules are being phased in; main duties start around May 2027. Consent, deletion and breach handling are built in from day one.
SMS reading, screen-scraping and bank-password flows are banned	Google Play restricts SMS permissions. Scraping is unsafe. Account Aggregator is the lawful route, scheduled as a later phase.

Change from v1	Why
Added: offline-first, Hindi and regional language support, Remote Config kill-switches, optional on-device receipt scan	Market gaps found in section 2.
Build order changed to a thin core loop first	v1 built nine phases before anything was usable. v2 ships "add a transaction in under 10 seconds" first.
Cloud Functions need the Blaze plan	Privileged operations (phone ownership, roommate acceptance) need them. Budget alerts and a Gemini spending cap are now required setup.

Nothing from v1 was removed: all 8 themes, the Cash-Flow Reactor, Dynamic Island, Indian currency engine, biometric lock, roommate ledger, CSV, legal screens and the three authors are still required.

Added in v2.1: a liquid glass design system with eight glass themes (section 14), a slot and a protocol for your own design (section 15), and a wave-based feature expansion with navigation, widgets, smart quick-add, subscriptions and an IOU ledger (section 16). Open decisions moved to section 17.

Added in v2.2: the owner-approved decisions, conflict resolutions, revised build order, offline rules and a merged decision register (section 18); the execution directive and required deliverables (section 19); and a starting audit with the exact safe-to-spend rules (section 20). Where section 18 and an earlier section differ, section 18 wins.

2. Market analysis and positioning

Fluxora should not try to beat payment giants. It should be the household money layer that sits on top of UPI: track, budget, split with flatmates, and settle through the UPI app people already use.

What the evidence says

- Finance app downloads in India are concentrated: PhonePe alone had 102.2M downloads in the year to April 2026, and the top five apps hold over a quarter of top-500 finance downloads (AppTweak). Competing on payments is a losing fight.

- Pure personal-finance software is a small market: about USD 42.5M in 2024, forecast to reach USD 63.6M by 2033 (IMARC). Growth will come from apps that are AI-assisted and mobile-first, so Fluxora must win on experience, not on market size alone.
- Splitwise's free plan caps daily expenses, which pushed Indian users to alternatives such as Tricount, Splid, Settle Up and Google Pay (Tricount comparison, News18 via ITVoice). Many of those only record debts and settle outside the app.
- Account Aggregator (AA) is now mature: 2.61 billion accounts were enabled for data sharing and 252.9 million users had linked accounts by 31 December 2025 (HyperVerge). Automatic bank data is the next big expectation, but it needs licensing steps (see section 5).

Gaps Fluxora targets (my analysis, not a survey)

User need	Gap in current apps	Fluxora answer
Log a spend fast	Most trackers need many taps	Quick-add sheet: amount, category chip, done in under 10 seconds
Split rent and bills without a paywall	Daily caps on free plans	No caps; four split modes; UPI handoff
Know who has really paid	Apps record debts but do not confirm payment	Two-party settlement confirmation
Trust with money data	Fear of bank-password scraping and ad tracking	No scraping, no ads, consent-based data, biometric lock, amount masking
Bills never missed	Reminders are generic	Recurring bills with one-tap "paid" and no duplicates
Understand, not just see charts	Charts without advice	One daily insight, explained on tap
Works on poor networks	Many split apps need constant connection	Offline-first with sync
Use in own language	English-only	Hindi plus regional languages through string resources

Positioning line: "Your household's money, simple and private."

Who it serves first: students and young professionals sharing flats in Indian cities, then couples and families. Start with them because rent, groceries, utilities and subscriptions create the shared-ledger need on day one.

3. Progressive disclosure and encapsulation (hard rule)

The user sees the minimum needed to decide or act. Everything else stays one tap away or hidden. The same idea applies to code: each part exposes a small public surface and hides its internals.

A. What the user sees (progressive disclosure)

Every screen follows three levels.

Level	Shows	Example
0, glance	One headline number, one insight, one primary action	Home: net balance this month, one insight line, an "Add" button
1, one tap	The detail behind the headline	Tap balance: income, expenses, top 3 categories
2, on demand	Full lists, filters, advanced options, raw data	"See all transactions", "More split options", export

Apply these rules everywhere:

1. **One primary action per screen.** Secondary actions go in an overflow menu or a bottom sheet.
2. **Smart defaults.** Date defaults to today, split defaults to equal, currency is INR, category is suggested from the last similar entry. Users change defaults only when they need to.
3. **Hide what is not relevant yet.** No roommate card until a roommate is connected. No budget card until a budget exists. Show a short empty state that teaches the first step.
4. **Lists show 5, then "See all".** Dashboard shows 5 recent transactions, 3 budget bars, 1 goal.
5. **Advanced behind "More options".** EXACT, PERCENTAGE and SHARES split modes, recurring rules, custom categories and CSV tools are not on the first screen.

6. **Dynamic Island shows one thing at a time**, only when the user can act on it (for example "Food budget at 85%"). Nothing to act on means it stays quiet.
7. **Notifications are limited and grouped.** At most one per category per day unless the user acts. Daily insight is one notification.
8. **AI answers lead with a 2-sentence summary.** A "Why this?" control expands the reasoning and the numbers used.
9. **Errors are human.** Show "Couldn't save. Check your connection and try again." Technical text and a support code sit behind a "Details" control.
10. **Never show internal data.** No document IDs, UIDs, raw timestamps, status enums or stack traces. Phone numbers appear masked (`+91 98••••••10`) except where the user is entering their own.
11. **Onboarding is 3 screens maximum.** Ask for notification, biometric and contact permissions in context, at the moment they are needed, with one line of why.
12. **A Simple / Advanced switch in Settings.** Simple is the default. Advanced reveals analytics, import tools and developer-style options.
13. **Amount masking** (section 10) is the user-facing privacy layer on top of these rules.

B. What the code exposes (encapsulation)

1. **Feature modules expose a tiny API**: one route/entry composable, one or two use-case interfaces, and read-only domain models. Everything else is Kotlin `internal` or `private`.
2. **Repositories are interfaces** in the domain layer. Firestore implementations are `internal` to the data layer and never imported by UI code.
3. **Firestore DTOs never leave the data layer.** Map to domain models at the boundary, then to display-ready UI state.
4. **`Money` is a value class** wrapping minor units. UI and ViewModels cannot do raw `Long` arithmetic on money; only `FinancialEngine` can.
5. **One immutable `UiState` per screen** plus one `onEvent(...)` function. Composables receive display strings, not domain objects.
6. **Privileged logic lives behind narrow callable functions** that return only the fields the caller needs, never whole documents.
7. **AI receives aggregates, not records** (section 9).
8. **Secrets stay out of the app.** No API keys in source, resources or BuildConfig.

9. **Modules cannot reach into each other.** A Gradle dependency rule or a lint check fails the build if a feature module imports another feature's `internal` code.

C. How the coding agent reports to you

Short answers by default: what was done, what passed, what is blocked, and the next step. Full detail goes into `FLUXORA_IMPLEMENTATION_STATUS.md`, not into chat.

4. Mission and working method

Act as one coordinated senior team: Principal Android Architect, Kotlin and Compose engineers, Firebase backend architect, security engineer, financial-systems engineer, UI/UX designer, AI integration engineer, performance engineer, QA automation engineer and release engineer.

Build **Fluxora**, a production-oriented native Android app (Kotlin, Jetpack Compose, Material 3) from a fresh project. It is not a UI prototype. It must have correct money arithmetic, server-enforced security, real backend integrations, automated tests and a verifiable build.

Fluxora in one line: a private household-money app for India: personal finance, budgets, savings, recurring bills, a roommate split ledger with UPI settlement, and an AI advisor.

Identity (keep exactly): app name Fluxora; `applicationId` = `com.aistudio.financetrack.qwrxnv`; `namespace` = `com.example`. These are separate Gradle properties. Change them only for a documented tooling reason and with my approval.

Source of truth and precedence

1. My instructions in the current conversation.
2. This specification.
3. Supporting documents I attach (such as `Fluxora_Complete_Summary.pdf`).
4. Verified existing project code.

If two requirements conflict, name the conflict and your resolution before implementing it. Never silently remove, reinterpret or downgrade a feature. Never reuse old Firebase configs, OAuth credentials, API keys or signing keys.

Principles

1. Correct money over animation. Working features over decoration.
2. Security is enforced on the server; the client is never trusted.
3. Show only essential details (section 3).
4. Verify dependency versions and model names before using them; never guess.
5. Never fabricate financial records, credentials, APIs, tests or builds.
6. A feature is not complete because a screen exists.
7. Preserve working code when changing things.

Workflow

1. **Inspect first.** List the project, Gradle files, manifest, sources, tests and Firebase config. Write an inventory. If the project is empty, initialise it.
2. **Work in phases** (section 13). For each phase: state the goal and files, implement it fully, compile, run relevant tests, fix errors, record evidence, update the status file, and only then move on.
3. **Do not stop after planning.** Continue building. If a credential is missing, finish the app-side integration, mark the external step "awaiting owner configuration", and carry on with independent work.
4. **Keep** `FLUXORA_IMPLEMENTATION_STATUS.md` with completed and pending phases, files changed, dependencies, external setup needed, compile and test results, known defects and the next task. Update it after every phase.
5. **If interrupted**, reinspect the real project state and resume from the last verified phase. Do not assume earlier work succeeded.
6. **File rules:** create real files; no duplicate classes, unresolved imports, placeholder functions or TODOs in features marked complete. If a file cannot be created, say so.

5. Tech stack and API decisions

Before writing code, verify Kotlin, AGP, Gradle, KSP, the Firebase BoM and the Firebase AI Logic SDK are compatible, then pin stable versions in a Gradle version catalog. Never invent version numbers. Do not use obsolete Firebase KTX artifacts.

Additions for glass effects, widgets, voice input and receipt scan (a blur and refraction library chosen in a spike, Jetpack Glance, Android speech recognition, on-device ML Kit) are specified in sections 14 and 16.

Core stack (unchanged from v1 except where noted)

Layer	Choice
Language, UI	Kotlin, Jetpack Compose, Material 3
Architecture	MVVM + Repository, Coroutines, StateFlow, Gradle modules per feature
Local preferences	Jetpack DataStore
Background work	WorkManager
Auth	Firebase Authentication; Google sign-in through Android Credential Manager; email/password; phone OTP
Database	Cloud Firestore with offline persistence on. No Room.
Privileged operations	Firebase Cloud Functions (2nd gen, callable)
AI	Firebase AI Logic with the Gemini Developer API (model rule below)
App protection	Firebase App Check: Debug provider in dev, Play Integrity in production
Biometrics	AndroidX Biometric
Observability	Crashlytics, Analytics (no financial values or personal data in events)
Messaging	Firebase Cloud Messaging for roommate and settlement alerts
New: remote switches	Firebase Remote Config for the AI model id, feature flags and kill-switches

API decisions

Use now

API	Used for	Condition
Firebase Auth, Firestore, App Check, Crashlytics, Analytics, FCM	Core app	Fresh Firebase project; no old credentials
Cloud Functions callable	Phone ownership, roommate acceptance, atomic bill payment, settlement confirmation	Needs the Blaze plan; set a budget alert
Firebase AI Logic + Gemini	Daily insight and chat	See AI model rule
Remote Config	Model id, flags	Ship safe defaults inside the app
Android UPI intent (`upi://pay`)	Settling roommate balances through the user's own UPI app	Personal payments only; see section 8
SMS Retriever API	Autofill of the Firebase phone OTP	Does not need SMS read permission
Android Photo Picker and on-device ML Kit text recognition	Optional receipt scan in Phase 7	Processing stays on the device; user confirms every field
App Links	Roommate invite links	Must open the exact request, never auto-accept

Add later, after validation

API	Why wait
Account Aggregator (AA) through a licensed gateway such as Setu, Finvu or OneMoney	Gives automatic bank data with user consent, but AA users are institutions called FIUs that must follow ReBIT API specifications and sandbox onboarding (Sahamati). Confirm that a personal-finance app can qualify, or can go through a technology provider, before promising it.

API	Why wait
A licensed payment aggregator (Razorpay, Cashfree, PayU)	Only needed if you want bank-verified settlement. It requires a business account, KYC and signed webhooks.
Play Console SMS exception for money management	Google lists SMS-based money management as a possible use, but it is subject to Play review and approval (Google Play policy). Apply only if auto-capture proves essential.

Never use

- Screen-scraping or asking for bank passwords.
- `READ_SMS`, `READ_CALL_LOG`, or an accessibility service to read other apps.
- Gemini or any API key inside the APK.
- UPI Collect or typed-VPA payment screens as the main flow.
- Room, or any second financial database.

AI model rule

Do not hard-code `gemini-2.5-flash`. Google's deprecation page lists it with a shutdown date (Gemini API deprecations), and developer reports say 2.5 models returned errors earlier than announced and were later limited to accounts that had used them before (Google AI developer forum). Instead:

1. Open the current Firebase AI Logic model list (Firebase docs) and choose a currently supported stable Flash-tier model.
2. Store the model id in Remote Config, with the chosen id as the in-app default.
3. Add one fallback model id, and fall back automatically on a "model not found" error.
4. Record the choice and the check date in the status file.

6. Architecture and data model

Module layout

Use Gradle modules so that encapsulation is enforced by the compiler, not by habit.

Module	Contains	Exposes to others
`:app`	`MainActivity`, navigation graph, dependency wiring	Nothing
`:core:money`	`Money` value class, `FinancialEngine`, `DateValidator`	Public engine API only
`:core:design`	Eight themes, typography, reusable components, `LocalHideSensitiveAmounts`	Theme and components
`:core:security`	Biometric, App Check setup, privacy state	Small interfaces
`:data`	Firestore implementations, DTOs, mappers, DataStore	Repository interfaces only; implementations are `internal`
`:domain`	Domain models, use cases, validators	Interfaces and models
`:feature:*`	dashboard, transactions, budgets, savings, recurring, roommate, payments, ai, notifications, settings, legal, authors	One route and one entry composable each
`:functions` (Node/TypeScript)	Cloud Functions	Callable endpoints with typed input and output

ViewModels own screen state. Composables only draw. No financial calculation inside a composable.

A single `FinanceRepository` interface (split into small per-area interfaces behind it) is the one entry point to financial data. Its Firestore implementation is `internal` to `:data`. For every Cloud Function, write one line in the status file explaining why it must run on the server (for example: phone ownership and roommate acceptance involve two users and cannot be trusted to a client).

Firestore design (define and document before coding)

All money is stored as `amountMinor: Long` in paise. Define `MAX_AMOUNT_MINOR` (suggested: 10,000,000,000 paise, i.e. 10 crore rupees) and enforce `1..MAX_AMOUNT_MINOR` in the app, the functions and the rules.

Path	Document id	Who can write	Notes
`users/{uid}`	uid	Owner, limited fields	Profile and preferences. No `phoneVerified` field the client can set.
`users/{uid}/transactions/{id}`	Client-generated UUID (this is the idempotency key)	Owner	Type INCOME or EXPENSE; no future dates
`users/{uid}/categories/{id}`	UUID	Owner	The 12 default categories are bundled constants with stable ids (for example `default_housing`), not stored per user. Only custom ones are stored.
`users/{uid}/budgets/{yyyyMM_categoryId}`	Month + category	Owner	One budget per category per month
`users/{uid}/savingsGoals/{id}` and `.../contributions/{id}`	UUID	Owner	Contributions can be reversed, never silently edited
`users/{uid}/recurringBills/{id}`	UUID	Owner, except `nextDueDate`	`nextDueDate` changes only through the paid-bill transaction
`users/{uid}/billPayments/{billId_dueDate}`	Bill + due date	Function or Firestore transaction only	Existence of this document blocks duplicate payment

Path	Document id	Who can write	Notes
`users/{uid}/notifications/{id}`	`relatedId`-based	App and functions	Used for deduplication
`phoneOwners/{hashOfE164}`	Hash of the number	Functions only	Maps a verified phone to one uid
`connectionRequests/{id}`	UUID	Create by sender; status changes by functions only	Pending, accepted, declined, cancelled
`households/{id}` with `members`	UUID	Functions create; members read	Created on acceptance
`households/{id}/sharedExpenses/{id}`	UUID	Members	Stores per-participant shares in minor units
`households/{id}/settlements/{id}`	UUID	Payer creates; receiver confirms through a function	State machine in section 8

For every collection also document: required fields and types, validation, query patterns, composite indexes, deletion behaviour and retention. Deploy the indexes file.

Firestore security rules

Write real, deployable rules that enforce: authentication; `request.auth.uid` ownership; type, range and required-field checks; immutable owner fields; allowed status transitions; household-member-only access; and least privilege. Client validation is only a convenience. Test rules with the Firebase Emulator Suite: unauthorized reads, writes, ownership change attempts and invalid transitions must all be denied.

Data minimisation by design

Store a hash of the phone number for lookup and show only a masked form in the UI. A roommate can read only the shared household records, never the other person's personal transactions, budgets or goals.

7. Core finance features

Build the "core loop" first: open the app, add a transaction in under 10 seconds, and see the dashboard update.

Money engine

`FinancialEngine` is the only place that does money arithmetic. It uses integer paise (`Money` value class), never `Float` or `Double`.

- `formatIndianCurrency(money, showSign, style)` uses lakh and crore grouping. Example: 12,345,678 paise formats as `INR 1,23,456.78`; 3,905,000 paise as `INR 39,050.00`. Support a symbol style (`₹`) for display and the `INR` prefix style for exports.
- Safe add, subtract and multiply with overflow detection (`Math.addExact` style); input parsing that rejects more than 2 decimals; negative amounts shown with a sign only when asked.
- Deterministic split allocation: shares are computed in paise, then the leftover paise are given one each to participants in a fixed order (for example sorted by uid). The shares always sum exactly to the total.

Transactions

- Types INCOME and EXPENSE. `amountMinor` in `1..MAX_AMOUNT_MINOR`. `DateValidator.validateTransactionDate` allows today and past dates and rejects future dates, using the device's time zone (default Asia/Kolkata).
- Add, edit, delete, search, date filter, category filter.
- The client generates the document id (UUID). Retrying a save writes the same document, so there are no duplicates.
- **Quick-add:** a bottom sheet with a number pad, category chips (last-used first) and a Save button. Note, date and account are under "More".

Categories

Twelve defaults: Housing, Food & Dining, Groceries, Transportation, Education, Shopping, Entertainment, Health, Bills & Utilities, Travel, Subscriptions, Other. Defaults are global app constants that cannot be edited or deleted. Custom categories are per-user; names are trimmed, 1 to 30 characters, and unique ignoring case across defaults and custom ones. Deleting a custom category moves its transactions to "Other" after confirmation.

Budgets and savings goals

- Monthly category budgets. Usage = sum of that month's EXPENSE transactions in the category. Alerts at 80% (warning) and 100% (exceeded), each at most once per budget, per month and per threshold. Editing or deleting a transaction recalculates usage; if usage falls back below a threshold the alert can fire again later.
- Savings goals with milestones at 25, 50, 75 and 100%. Record the highest milestone already notified; a reversed contribution that drops below a milestone resets it. Contributions are tracked separately from expenses.

Recurring bills

Weekly, monthly and yearly. Month-end rule: a bill due on the 31st falls on the last day of shorter months; a 29 February yearly bill falls on 28 February in non-leap years.

`markBillAsPaid` runs as one Firestore transaction (or a callable function) that:

1. Checks that `billPayments/{billId_dueDate}` does not exist; if it does, returns the earlier result without writing anything.
2. Creates the expense with a deterministic id built from the bill and due date.
3. Creates the `billPayments` record.
4. Advances `nextDueDate`.

A 10-second debounce in the UI is kept as a courtesy, but the deterministic ids are the real protection against duplicates.

Dashboard: exact definitions

Document these in code comments and the status file.

Metric	Definition
Total income / expenses	Sum of INCOME / EXPENSE in the selected period (default: current month)
Net position	Income minus expenses for the selected period
Balance	Opening balance (optional, default 0) plus all-time income minus all-time expenses
Available to spend	Balance minus the total saved in goals
Savings overview	Sum of goal contributions and nearest milestone

Metric	Definition
Budget overview	The three budgets with the highest used percentage
Insight	One AI or rule-based sentence; never invented figures

The dashboard recalculates when data changes, respects the date filter and amount masking, and shows loading, empty and offline states. It does not display figures it cannot derive from the user's records.

8. Roommate ledger and UPI settlement

Split modes

EQUAL, EXACT, PERCENTAGE and SHARES. EQUAL is the default; the other three sit behind "More split options". All allocation is in paise and must sum exactly to the expense.

- **EQUAL:** base share = total divided by participants; the leftover paise go one each to the first participants in a fixed order (sorted by uid).
- **EXACT:** the entered amounts must sum to the total, or the form shows how much is left.
- **PERCENTAGE:** percentages are held in basis points and must sum to 10,000. Use the largest-remainder method for rounding, with a uid tie-break.
- **SHARES:** positive integer weights, allocated by the same largest-remainder method.

Validate: participants are household members, no duplicates, positive shares, valid percentages, and the payer is a participant or is explicitly marked as paying for others. Show four different numbers, never blended: **amount owed**, **amount paid**, **outstanding balance** and **settled amount**. An optional "simplify debts" view sits at Level 2.

Connection requests

`RoommateConnectionRequest` with `isIncomingFor(uid)` and `isOutgoingFor(uid)`.

Action	Who	How it runs
Create	Sender with a verified phone	Callable function `createConnectionRequest`. Rejects self-requests and duplicate pending requests (by uid and by phone hash).
Accept	Recipient only	Callable `respondToConnectionRequest`: confirms the request is PENDING, both phones are verified, creates the household membership and updates the request in one transaction. A second accept does nothing.
Decline	Recipient only	Same function; PENDING to DECLINED.
Cancel	Sender only	Callable `cancelConnectionRequest`; PENDING to CANCELLED.

Privacy rule: when a user searches by phone number, never reveal whether that number has an account. Reply "If this number is on Fluxora, they'll see your request." An unverified phone can never create or accept a request. `syncRemoteConnectionRequest` may only apply changes that a function already made.

UPI settlement handoff

`PaymentManager` builds a standard personal-payment intent: `upi://pay?pa=<VPA>&pn=<name>&am=<amount, 2 decimals>&cu=INR&tn=<note>`. It is for person-to-person settlement only. Merchant or collection flows need a licensed payment aggregator and are out of scope.

Why no typed VPAs in the main path: NPCI guidance effective February 2026 restricts manual VPA entry on mobile and makes intent the standard flow (Razorpay on NPCI guidelines). So the user confirms a **saved payee** once (VPA validated by pattern and shown masked), and later settlements need one tap. Offer QR scan as an alternative way to add a payee.

- Validate the VPA pattern and the amount (positive, at most 2 decimals, within `MAX_AMOUNT_MINOR`). Do not hard-code bank transaction limits.
- Use the Android app chooser. If no UPI app exists, say so and offer "Copy VPA".
- Handle cancel and failure results. Treat any result data from the UPI app as a hint only, because it can be missing or forged.

Settlement states

Launching a UPI app is not proof of payment. Never mark a settlement as paid because an intent opened or returned.

State	Meaning	Set by
PENDING	Balance owed, nothing started	System
INITIATED	Payer opened a UPI app	Payer's device
USER_CANCELLED / FAILED	Payer backed out, or the app reported failure	Payer's device
CONFIRMATION_REQUIRED	Payer says "I paid"; waiting for receiver	Payer, through a function
CONFIRMED_BY_RECEIVER	Receiver confirms the money arrived; the balance is settled	Receiver only, through a function
DISPUTED	Receiver says not received	Receiver, through a function
VERIFIED_SUCCESS	Reserved. Only for a future bank-verified provider with signed confirmations	Provider webhook

The UI says "Settled (confirmed by both)" for CONFIRMED_BY_RECEIVER. It never says "bank verified". Transitions are validated in the function and the Firestore rules.

9. AI financial advisor

Use Firebase AI Logic with the Gemini Developer API and the model rule in section 5. Features: a daily saving insight, `GeminiChatDialog`, `GeminiChatViewModel` (with `resetUiState`), and answers that are aware of the user's budgets, bills and spending.

What the user sees

- Home shows **one** insight sentence. Tap it for a short explanation and the numbers used. Chat opens from a button, not by default.
- Every AI answer leads with a 2-sentence summary; "Why this?" expands details.
- A small line states that AI output is informational help and not financial advice.

- If AI is unavailable, the insight falls back to a rule-based sentence computed from the user's own data (for example "Food spending is 22% above last month"). It must be true, and it must say nothing if there is nothing true to say.

What the model receives (data minimisation)

1. Send **aggregates**: category totals for the current and previous month, budget percentages, upcoming bill names and due dates. Do not send transaction notes, payee names, phone numbers, emails, UIDs or full transaction lists.
2. Never send roommate or household data that belongs to another person.
3. Treat all user-entered text (notes, category names) as untrusted. Pass it as data, never as instructions, and ignore instructions found in it (prompt injection).
4. AI features need a one-time opt-in consent screen in plain language, with an off switch in Settings. Check how Google's terms treat prompts for your chosen tier, and state the result in the privacy policy.
5. The model may not invent transactions or figures. When asked something the data cannot answer, it says so.

Reliability

- 30-second timeout, retry of transient errors with backoff, and correct cancellation.
- Friendly messages for no network, quota exceeded, App Check failure and "model not found" (which triggers the fallback model).
- Cache the daily insight and generate it at most once a day. Cap chat requests per user per day, and set a Gemini spending cap and budget alerts in the Firebase and Google Cloud consoles.
- Log only error categories to Crashlytics, never prompts or answers.
- Never disable App Check to make an AI call succeed.

10. Security, privacy and compliance

Authentication and phone ownership

- `AuthManager` supports Google (Credential Manager), email/password, and phone OTP. Firebase Auth is the identity authority; never trust a UID or verification flag sent by the client.

- **Phone ownership** runs in a callable function `reservePhoneOwnership`. The function reads the verified phone number from the caller's ID token, normalises it to E.164, and in one transaction writes `phoneOwners/{hash}` to the caller's uid. If another uid already owns the number, it refuses. Define behaviour for number change (release the old record), account deletion (release it), expiry and failure.
- `PhoneValidation.kt` and `CountryPhoneInputField.kt`: searchable country list, India (+91) default, national-digit extraction, E.164 output. `PreferencesManager` exposes `isPhoneVerifiedFlow`, `getVerifiedPhoneFlow` and `getPhoneOwnerFlow`; these mirror server state and never decide it.

App lock and privacy

- `BiometricAuthManager`, `BiometricAuthScreen`, `AppLockMonitor` using AndroidX Biometric with device-credential fallback. Configurable inactivity timeout; lock when the app goes to the background; clear session state on sign-out. Never simulate biometrics and never store face images.
- Hide balances in the recent-apps preview and block screenshots on sensitive screens (secure window flag), with a Settings toggle.
- `LocalHideSensitiveAmounts` is one central CompositionLocal backed by DataStore. It masks amounts on the dashboard, history, budgets, savings, roommate ledger, analytics, AI summaries, Dynamic Island and notification text.

App Check

Debug provider in development, Play Integrity in production, with separate configuration. Never ship debug tokens. Handle invalid-token errors with a clear message. Never turn App Check off to make a request pass.

Data protection under India's DPDP law

India's Digital Personal Data Protection Rules were notified on 13 November 2025 with phased start dates. Consent-manager registration begins around November 2026, and the main duties (notice, consent, security safeguards, breach reporting, deletion, children's data) start about 13 to 14 May 2027 (TaxGuru, Consent.in, AMSS regulatory alert). Confirm exact dates in the Gazette text before launch. Build these in now:

1. A plain-language privacy notice and consent screen before data is collected, with separate consent for AI processing and for contacts or notifications.
2. A consent log (what, when, which version) and a way to withdraw consent.

3. In-app account and data deletion, plus a data-export option. Google Play also expects an in-app account deletion path for apps that create accounts; confirm in the Play Data safety form.
4. A breach-response runbook. Reports describe a short notification deadline (72 hours is commonly cited); confirm in the Rules.
5. An 18-plus age declaration, because children's data triggers parental-consent duties.
6. Retention rules for each collection, written down.
7. Analytics events carry no amounts, names, phone numbers or notes.

Never claim end-to-end encryption. Say what is true: data is encrypted in transit, and Google Cloud encrypts Firestore at rest.

Permissions policy

Ask only for notifications (Android 13+ runtime permission), biometrics, and contacts only if a contact picker is built. No SMS read, call log, location or storage-wide access. Use the system file picker for CSV and the Photo Picker for receipts.

11. Signature experience and performance

The look is a premium, calm fintech interface: strong hierarchy, generous spacing, restrained gradients, meaningful empty states, and clear loading and error states. Essential-first rules from section 3 apply to every screen below.

Eight themes

Midnight Emerald (default), Cyber Violet, Solar Gold, Arctic Cyan, Crimson Pulse, Obsidian Rose, Aurora Borealis, Monochrome Slate. Each has a light and a dark mode and defines primary and secondary colours, backgrounds, surfaces, text, navigation, cards, dialogs, chart colours and component states. All live in `Theme.kt`. Selection is stored in DataStore, applies to the whole app at once, and survives restart. Check text contrast of at least 4.5:1 for body text in all 16 combinations, and never use colour alone to show meaning (add icons or labels for over-budget and negative values).

Glass tokens, the eight glass colour sets, the quality tiers and the glass components are specified in section 14. The Dynamic Island, bottom bar and Reactor backdrop follow that system. If you provide a design (section 15), its colours and shapes override these defaults.

FinancialHero3DScene (Cash-Flow Reactor)

A Canvas-based interactive scene, not an image: an orbital core, touch-drag rotation, animated particle streams, income rings and expense rings, and live net-balance telemetry from real data.

- **Disclosure role:** this is a Level 0 visual. It shows only net position and the income/expense balance. Tap it to open the Level 1 breakdown. Amounts follow the masking setting.
- **Performance:** pre-allocate particle arrays and `Paint` / `Path` objects; no allocation inside the draw loop; drive animation from frame time; keep state reads inside the draw scope so the screen does not recompose each frame; stop animating when the app is in the background or the scene is off screen.
- **Adaptive quality:** measure frame time and reduce particle count and ring detail on slower devices. Honour the system "remove animations" setting by showing a still, readable version.
- **Accessibility:** a `contentDescription` states net balance, income and expenses in words (or "amounts hidden").
- **Evidence:** aim for the display's native refresh rate, including 120 Hz where supported. Measure with Macrobenchmark and JankStats, record frame-time results on at least one mid-range and one low-end device or emulator profile, and report the numbers. Do not claim a frame rate without a measurement.

DynamicIsland

A floating top capsule driven by real state: cash-flow health, active budget warnings, and quick actions. It shows one item at a time and only when there is something to act on ("Food budget at 85%", "Rent due tomorrow"). It animates smoothly, is reachable by screen readers, respects masking, and never shows invented or stale alerts. It handles display cutouts and small screens.

Layout and devices

- Call `enableEdgeToEdge()`. Apply Scaffold padding exactly once; handle status and navigation bar insets, display cutouts, keyboard overlap (`imePadding`), dialog clipping, text overflow and floating button collisions.
- Design first for a 360 dp wide phone with large font settings; then support tablets and foldables with window size classes.

- Choose the lowest `minSdk` that the verified dependencies allow (suggested target: API 26), because many users in India are on budget phones. Record the choice and the reason.
- Support Hindi and at least one more Indian language through string resources, with Indian digit grouping and the rupee symbol in every locale. Keep text expansion in mind for layouts.

12. Supporting features

Notifications

`NotificationManager`, `SystemNotificationHelper`, `BillReminderWorker`, `BillActionReceiver`. Five categories: Budget, Bill, Savings, Roommate, Insight, each with its own Android channel.

- A master switch and one switch per category. Turning the master off suppresses everything but keeps the category choices, so turning it back on restores them.
- Deduplicate by `relatedId` (for example budget id + threshold + month). Cancel or update a notification when its cause changes.
- Ask for the Android 13+ notification permission at the first moment it is useful, not at launch.
- WorkManager timing is approximate. Do not promise exact wall-clock delivery, and compute times in the user's time zone.
- The "Mark paid" action on a bill notification calls the same atomic `markBillAsPaid` path as the app.
- Respect masking: when amounts are hidden, notification text omits them.
- Limit volume: at most one notification per category per day unless the user acts; the daily insight is one notification.

CSV export and import

Export (`exportToCsv`): UTF-8, columns `id, date (yyyy-MM-dd), type, category, amount (rupees, 2 decimals), note`. Escape commas, quotes and line breaks. Neutralise formula injection: any text cell that starts with `=`, `+`, `-`, `@`, a tab or a carriage return is prefixed so spreadsheets read it as text. Numeric columns are exempt. Offer an option for UTF-8 with a byte-order mark so Excel shows Hindi text correctly.

Import (`importFromCsv`):

1. Pick the file with the system picker; set a sane size and row limit.
2. Validate required columns, amounts, dates (no future dates), types and categories.
3. Show a **preview** with counts: valid rows, rows with errors (with row numbers and reasons), and duplicates (within the file and against existing transactions).
4. Commit only after the user confirms, and only the valid, non-duplicate rows. Never import malformed rows silently.
5. Show a short summary afterwards.

Legal screens

`LegalScreens.kt` holds the Privacy Policy and Terms & Conditions. The Privacy Policy has eight accordion sections and must describe what the app actually does: Firebase services used, AI processing with Gemini, what is collected and why, retention, user rights, deletion and export, and how to complain or contact. No generic placeholder text. Contact links go through `openContactEmail`.

Official legal contacts are Ayush Mishra and Sunaina Bansal. **Do not invent their email addresses.** Read the addresses from deployment configuration, and ask me for them if they are missing. Legal text must be updated whenever data handling changes.

Authors screen

`AuthorsScreen.kt` shows exactly three cards, with the names and links below, opening each link externally. No invented job titles, no duplicates.

Author	LinkedIn URL (use exactly)
Ayush Mishra	https://www.linkedin.com/in/ayush-mishra-12b36a37a/
Sunaina Bansal	https://www.linkedin.com/in/sunaina-bansal-b5107a374/
Rohan Kumar	https://www.linkedin.com/in/rohan-16kumar

Settings

Simple by default: theme, hide amounts, app lock, notifications, language, AI on/off, legal, authors, sign out. An **Advanced** switch reveals lock timeout, CSV tools, data export, account deletion details and diagnostics.

13. Testing, roadmap and completion

Roadmap (core loop first)

Finish each phase's exit criteria, with evidence, before starting the next.

Phase	Scope	Exit criteria
1. Foundation	Fresh project, Gradle modules, version catalog, ids, navigation, status file	Clean build; module boundary check passes
2. Money and design	`Money`, `FinancialEngine`, `DateValidator`, eight themes, DataStore, reusable components	Engine tests pass; all themes switch and persist
3. Core loop	Auth, Firestore, rules, transactions, categories, quick-add, basic dashboard	Adding a transaction takes under 10 seconds in a UI test; rules emulator tests pass
4. Planning	Budgets, savings goals, recurring bills with atomic payment, notifications	Duplicate-payment and alert-dedupe tests pass
5. Identity and security	Phone ownership functions, biometric lock, amount masking, App Check	Unauthorized access and takeover tests denied; masking verified on every screen
6. Household	Split ledger, connection requests, saved payees, UPI handoff, settlement states	Splits sum exactly; invalid transitions denied; no "paid" state from an intent alone
7. Experience and AI	Hero scene, Dynamic Island, AI insight and chat, Remote Config model switch	Frame-time numbers recorded; AI error paths and fallback tested
8. Supporting	CSV, legal, authors, settings, languages, optional receipt scan	CSV injection tests pass; three author URLs verified; deletion and export work
9. Audit and release	Performance, accessibility, security review, regression, debug APK	No open critical defects; the APK file exists and its path is verified

If a credential is missing, finish the app side, mark the item "awaiting owner configuration", and continue with independent work.

v2.1 additions to the phases: Phase 2 also includes the glass spike, the `:core:glass` module and, if a design is provided, the design inventory `FLUXORA_DESIGN_MAP.md` (sections 14 and 15). Wave 1 features from section 16 fit as follows: smart quick-add and safe-to-spend in Phase 3; subscriptions and rent templates in Phases 4 and 6; widgets, shortcuts, IOU ledger and weekly review in Phase 8. Phase 9 adds screenshot tests for every theme in every glass tier. Wave 2 and Wave 3 wait until Phase 9 passes.

Tests to write and run

The 74 tests and the successful build in the old summary are history only; they prove nothing about this new project. Never say a test passed unless it was run.

- **Money engine:** Indian grouping (for example `INR 1,23,456.78`), overflow, negatives, remainder allocation, balances.
- **Transactions:** validation, future-date rejection, idempotent retry, summaries.
- **Security (emulator):** unauthorized reads and writes, ownership change attempts, phone ownership races, invalid roommate and settlement transitions.
- **Recurring bills:** atomic payment, retry, duplicate prevention, month-end and leap-year dates.
- **CSV:** escaping, formula injection, bad rows, duplicates, preview and summary.
- **Payments:** VPA and amount validation, no UPI app, cancel, forged or missing result data, state machine.
- **Notifications:** master and category switches, dedupe, permission denied.
- **UI:** theme persistence, masking everywhere, navigation, small-screen layout, loading and error states.
- **Essential-first checks (new):** a test that scans rendered screens for UIDs, raw enums and stack traces; Level 0 screens show one primary action; no feature module imports another's `internal` code.
- **Privacy (new):** account deletion removes user data and releases the phone record; data export works; analytics events contain no amounts or personal text.

Completion states

Track every feature as NOT STARTED, IN PROGRESS, IMPLEMENTED, LOCALLY VERIFIED, BACKEND VERIFIED, COMPLETE or BLOCKED. A feature is COMPLETE only when its acceptance criteria pass. A screen existing is not completion. Record required behaviour, files, data dependencies, security needs, error handling, tests, the real result and blockers.

Final report (short)

Use these ten headings, one to three lines each unless something failed: overall status, completed, partial, pending owner configuration, security verification, test results, build results, APK location, known issues, next actions. The overall status is exactly one of: BUILD SUCCESSFUL; COMPILATION SUCCESSFUL, APK BUILD FAILED; PARTIALLY IMPLEMENTED; BUILD BLOCKED. Details go in the status file.

Non-negotiable rules

1. Fresh project; never reuse old credentials, keys or Firebase projects.
2. Keep every specified feature: eight themes, Cash-Flow Reactor, DynamicIsland, three authors with exact URLs, full legal screens.
3. Never fabricate financial data, APIs, test results or builds.
4. Never trust client-side authorization; privileged operations run on the server.
5. Never claim payment success without both-party confirmation or a trusted provider.
6. Never simulate biometrics; never expose secrets in source.
7. Show only essential details to the user; keep internals private.
8. Fix real compiler errors; do not remove features to make code compile.
9. Never claim unmeasured performance or unexecuted tests.
10. Keep `FLUXORA_IMPLEMENTATION_STATUS.md` current.

14. Liquid glass design system

Goal: Fluxora should look like light moving through glass over living colour, and still be fast and readable on a budget phone. Android has no built-in liquid glass API, so the effect is built from blur, colour controls and shaders, in tiers.

What the platform allows

Capability	Minimum Android	Source
Backdrop blur, colour controls (`RenderEffect`)	API 31	Kashif-E/AndroidLiquidGlass
Lens refraction, vibrancy, custom shaders (AGSL `RuntimeShader`)	API 33	same
Frosted fallback	API 25 and up in at least one library	PrismalAGSL releases

Libraries exist, but most are young: a Compose multiplatform port is at `0.0.1-alpha02`, and PrismalAGSL is a first stable release. Haze (Chris Banes) is the established blur library and its own docs say it adds measurable frame-time cost (Haze docs index). So:

1. In Phase 2 run a **glass spike**: build one card, one bottom bar and one sheet with (a) Haze for blur, (b) Kyant's AndroidLiquidGlass or PrismalAGSL for refraction, and (c) a minimal own AGSL shader. Check licence, last release date, API level, and frame time on a mid-range and a low-end profile.
2. Pick one, pin its version, and wrap it behind our own `GlassSurface` API so the library can be swapped without touching screens.
3. Never depend on an alpha library for core screens without the fallback below.
4. Apple's "Liquid Glass" is Apple's design language. Take inspiration from the material qualities only. Do not use Apple's logos, SF Symbols or fonts.

Design principles

1. **Glass is for floating layers, not everything.** Use it on the bottom bar, Dynamic Island, quick-add sheet, dialogs, and cards over the hero. Long lists and text-heavy screens sit on solid or lightly tinted surfaces.
2. **Two layers of glass at most** on any screen region. Glass on glass turns to mud and costs frame time.
3. **Colour comes from behind.** Each theme has an ambient backdrop (slow-drifting soft colour fields, or the Cash-Flow Reactor), and the glass picks it up.
4. **Readability beats effect.** Key figures and labels get an adaptive scrim so contrast stays at least 4.5:1 over the worst backdrop colour. No small body text over refractive edges.

5. **Meaning is never theme colour alone.** Income, expense, warning and danger use fixed semantic colours plus icons, because a theme like Crimson Pulse has a red accent that must not read as "expense".
6. **Essential-first still applies** (section 3). Glass makes screens calmer, not busier.

Three quality tiers (chosen automatically)

Tier	When	What it draws
Full	API 33+, enough memory, battery saver off	Backdrop blur, vibrancy, edge refraction, moving specular highlight, soft inner glow, press deformation, depth shadow
Soft	API 31 to 32, or Full tier running slow	Backdrop blur, saturation boost, tint, bright edge stroke, shadow
Frost	API 26 to 30, low-RAM device, battery saver, or user choice	Opaque tinted surface with fine noise and edge stroke. Same layout and shapes

The tier is chosen at runtime from the API level, `isLowRamDevice`, battery saver state and measured frame time, and it can step down on its own. Settings has "Glass effects: Auto, Full, Reduced, Off". Never let a tier change the layout.

Components (`:core:glass`)

`GlassSurface`, `GlassCard`, `GlassBottomBar` (a floating pill with a liquid bubble that stretches to the selected tab), `GlassSheet`, `GlassDialog`, `GlassButton`, `GlassChip`, `GlassToggle`, `GlassSlider`, `GlassProgress`, `GlassTopCapsule` (Dynamic Island), `GlassFab`. All read a `GlassTokens` object from the active theme: tint, blur radius, saturation, refraction strength, specular intensity, edge gradient, shadow, noise and corner shape.

Motion and feel

- Spring physics for press, morph and sheet movement (use Material 3 Expressive motion if the verified Compose BOM provides it).
- Press makes the glass bulge slightly and brightens the edge. Selecting a tab stretches and settles the bubble.
- The specular highlight follows a slow light source, or the device tilt if the user allows it. It is static when "remove animations" is on.

- Light haptic ticks on tab change, toggle and milestone. Respect the system haptics setting.

Eight glass themes

Colours below are starting values. **If the design in section 15 gives colours, those win.** Every theme has dark and light variants; light variants use pale tinted backdrops with white glass.

Theme	Mood	Dark backdrop	Accent	Glass tint
Midnight Emerald (default)	Calm, wealthy	`#04120E` to `#0B2A22`	`#34D399`	Emerald at 14%
Cyber Violet	Electric, night city	`#0B0420` to `#241047`	`#A78BFA`	Violet at 16%
Solar Gold	Warm, confident	`#1A1204` to `#3A2A08`	`#FBBF24`	Amber at 14%
Arctic Cyan	Cold, crisp	`#03161C` to `#06303B`	`#22D3EE`	Cyan at 13%
Crimson Pulse	Bold, energetic	`#1A0508` to `#3B0A14`	`#F43F5E`	Rose red at 14%
Obsidian Rose	Luxe, soft	`#0D0A0C` to `#241319`	`#FB7185`	Rose gold at 12%
Aurora Borealis	Animated, vivid	`#04101F` with drifting green, teal and violet fields	`#34D399` and `#8B5CF6`	Cool white at 12%
Monochrome Slate	Quiet, neutral	`#0B0D10` to `#1C2128`	`#E5E7EB`	White at 10%

All eight must switch live, persist, and keep the same layouts. Theme preview cards in Settings are real glass cards over each theme's backdrop.

Performance and accessibility rules

- Budget: at most 3 live blur surfaces per screen. Do not blur every list row; blur one header or footer instead. Downsample the backdrop and cap blur radius per tier.

- Pause backdrop animation off screen, in the background, and while a list is flinging.
- Dialog and sheet background blur can use the window blur APIs where cross-window blur is enabled, with the Frost tier otherwise. Verify support at runtime.
- Measure with Macrobenchmark and JankStats on mid-range and low-end profiles; report numbers per tier. Do not claim a frame rate without a measurement.
- Honour font scale up to 200%, 48 dp touch targets, TalkBack labels, and "remove animations". Add an in-app "Increase contrast" option that raises tint opacity and edge strokes.
- Screenshot tests (Paparazzi or Roborazzi) for each theme in each tier on the main screens.

15. Provided design (owner fills this in)

You said you will give a design. Put it here, or paste it into the same message as this prompt. The coding agent must treat it as the visual source of truth.

Design input	Your content (edit in place)
Design files or links (Figma, images, PDF)	*to be added*
Screens covered (list them)	*to be added*
Colours, gradients, glass values (if you have them)	*to be added*
Fonts	*to be added*
Logo and app icon	*to be added*
Things you do NOT want changed	*to be added*
Things the agent may improve	*to be added*

How the agent must use the design

1. **Precedence for how it looks:** your design first, then section 14, then v1 visual defaults. Your design never overrides security, money correctness or the essential-first rules (section 3).
2. **Inventory first.** Before coding, list every screen in the design, every component, every state (loading, empty, error, masked amounts) and every colour, font and spacing value. Save it as `FLUXORA_DESIGN_MAP.md`: screen, composable, data it needs, states.

3. **Extract tokens, never hard-code.** Colours, radii, spacing, type scale and glass values go into the theme and `GlassTokens`. Screens only read tokens.
4. **Fill the gaps in the same style.** Screens the design does not show (settings, legal, errors, empty states) follow its patterns. List them in the design map so you can review them.
5. **Conflicts are reported, not silently fixed.** If the design fails contrast, cannot run at the target frame rate, or contradicts a rule here, keep the layout, make the smallest change that fixes the problem, and list it in the status file with the reason.
6. **Check fidelity.** Take screenshot tests of each designed screen and compare them against the reference. Report differences instead of claiming a match.
7. **Own your assets.** Use only fonts, icons and images you have the right to use. Do not copy another company's logo, icons or brand assets, and do not use Apple's SF Symbols or SF fonts.
8. **If no design arrives,** build from section 14 and the layouts in sections 3 and 11, and say so in the first status update.

Questions the agent may ask once, in a single batch

Which screen order matters most, whether the design is dark-first or light-first, and whether the bottom bar is a floating pill or a full-width bar. If unanswered, use the design as shown, or a floating pill when it is not shown.

16. Feature expansion

These features are added on top of everything in sections 7 to 12. They follow the same rules: essential-first (section 3), no secrets or raw data in the UI, and a small public API per module. **Rule: no extra feature may delay the core loop.** Build Wave 1 inside the phases in section 13; Wave 2 and Wave 3 start only after Phase 9 passes.

Why these features

Competing money apps commonly offer subscription detection, net-worth tracking and automatic categorisation (Retirewise, India, 2026). AI input by typing or voice, receipt scanning and batch receipt capture are now standard selling points (Finny comparison). Receipt scanning, a subscription manager and home-screen widgets appear together in newer trackers (Pocket listing), and widgets for fast logging and a balance check without opening the app are a selling point in others (NETO listing). The India-specific items (IOU ledger, Hinglish input, rent templates) are my proposals, not survey results.

Feature list

Feature	What it does	Wave
Smart quick-add	One text box: "chai 40" or "kirana 850 yesterday". An on-device parser fills amount, category and date; the user sees chips and taps Save. Nothing is saved without a tap. Understands common Hindi and Hinglish words.	1
Safe to spend today	One number: (remaining monthly budget minus bills still due this month) divided by days left, never below zero. Shows a dash if no budget exists. Deterministic, inside `FinancialEngine`.	1
Home-screen widgets	Jetpack Glance widgets: quick-add button, "safe to spend today" plus next bill. They respect amount masking and show nothing sensitive when the phone is locked. Glance cannot blur, so widgets use the Frost look.	1
Shortcuts and tile	App shortcuts and a Quick Settings tile for "Add expense".	1
Subscription manager	Finds likely subscriptions from the user's own transactions (3 or more similar amounts at weekly, monthly or yearly gaps), asks the user to confirm, then converts them into recurring bills. Shows monthly and yearly cost and reminds 3 days before renewal. No auto-cancel.	1
Rent and shared-bill templates	A recurring shared expense (for example rent on the 1st, split by percentage) that is created once per due date through the same idempotent function as bills. Everyone gets one notification.	1
IOU ledger	Track money lent or borrowed with people who are not on Fluxora: name, amount, due date, settled or not. A reminder is shared through the system share sheet with an optional UPI link. Stays private to the user. Not a loan product.	1
Weekly review	One Sunday summary notification with one suggested action. Can be turned off.	1

Feature	What it does	Wave
Voice quick-add	Speak the same sentence as smart quick-add, in Hindi or English, using Android speech recognition (on-device where available). Audio is never stored or uploaded by Fluxora.	2
Receipt scan	Photo Picker or camera, ML Kit text recognition on the device, prefills amount, date and merchant. The user confirms. The photo stays on the device unless the user chooses to attach it.	2
Spending forecast	"On track to spend about X by month end" from the run rate so far, with the calculation one tap away. The AI may explain it; it does not compute it.	2
Manual accounts and net worth	Cash, bank, wallet, card, with a net-worth trend. Appears only after the user adds a first account.	2
Monthly report card	A shareable image with amounts hidden by default, and a year-in-review in December.	2
Trip and event groups	Temporary groups joined by invite link, with the same split modes and settlement states. Needs its own privacy and abuse review first.	3
Account Aggregator import	Consent-based bank data through a licensed gateway, if eligibility is confirmed (section 5).	3

Navigation (a default; the provided design wins)

Five places in a floating glass bar: **Home, Activity, Add (centre), Household, Plan.** Plan holds budgets, savings goals, bills and subscriptions. The AI guide is a button on Home. Settings, IOUs and legal sit under the profile icon. Anything that is not needed on day one stays hidden until it is relevant.

System polish

Predictive back, themed app icon, per-app language (Android 13+, so Hindi works without changing the phone language), tablet and foldable layouts, and haptic feedback that respects system settings.

Do not build

Ads, selling or sharing user data, loan or credit marketing, crypto trading, automatic subscription cancellation, reading SMS or notifications from other apps, and any score or ranking of friends' payment behaviour.

17. Open decisions for the owner

The coding agent should continue with the default shown if a question is unanswered, and record the assumption in the status file.

Section 18 holds the merged, current decision register. If it differs from the table below, section 18 wins.

#	Question	Why it matters	Default if unanswered
1	What are the official contact emails for Ayush Mishra and Sunaina Bansal, and a support address?	Legal screens and the DPDP grievance contact need real addresses	Read from deployment config; show a "contact not configured" state, never a made-up address
2	Who is the legal data fiduciary (a person or a company)?	DPDP duties and the Play listing name a responsible party	Placeholder field in config; release blocked until filled
3	Keep `namespace` as `com.example`?	It looks like a template value, though it is separate from the application id	Keep as specified
4	Is the Blaze plan with billing acceptable, and what monthly budget cap?	Cloud Functions need it; Gemini paid usage follows	Set a budget alert and a Gemini spending cap before launch
5	Which languages after English and Hindi?	Translation effort and layout testing	Add one more later; keep all strings in resources now
6	Do you want Account Aggregator, a payment aggregator or an SMS exception later?	Each needs a business entity, onboarding and review	Not in v1; revisit after the core loop has real users

#	Question	Why it matters	Default if unanswered
7	Free forever, or a paid tier later?	Competitors lost goodwill with daily caps on free plans	No caps in v1; decide monetisation after launch
8	Accept the dashboard definitions in section 7 (balance, available to spend)?	Every figure on screen depends on them	Accept as written
9	Is API 26 as a target `minSdk` acceptable?	Wider reach on budget phones vs newer APIs	Lowest level the verified dependencies allow

Added questions (v2.1)

- **10. Design:** Will you provide design files (section 15)? Default: build from section 14.
- **11. Glass library:** May the agent choose the glass library after the spike? Default: yes, behind our own `GlassSurface` wrapper.
- **12. Wave 1 scope:** Ship Wave 1 features in the first release, or core only? Default: Wave 1 after the core loop is verified.
- **13. Voice and receipts:** Keep audio and receipt photos on the device only? Default: yes.
- **14. Tilt highlight:** Allow the glass highlight to follow the motion sensor? Default: off until the user opts in.

Owner setup checklist (the agent must document each step and mark it "awaiting owner configuration" until done)

1. Create a new Firebase project and register the Android app with the exact application id.
2. Download `google-services.json` (never invent or commit someone else's).
3. Enable Authentication providers: Google, email/password, phone. Configure the Google OAuth client and SHA fingerprints.
4. Create the Firestore database; deploy the security rules and the indexes file.
5. Enable Firebase AI Logic with the Gemini Developer API; set the Remote Config model id and fallback.
6. Register App Check: Debug provider for development, Play Integrity for production.
7. Enable Crashlytics, Analytics and Cloud Messaging.
8. Move the project to the Blaze plan; set budget alerts and a Gemini spending cap.

9. Deploy the Cloud Functions and run the emulator test suite.
10. Supply the legal contact emails and the data fiduciary details from section 17.
11. Complete the Play Console Data safety and account deletion declarations.

Track each external item as: implemented, locally verified, backend configured, awaiting owner configuration, or blocked.

Sources

- AppTweak: most downloaded finance apps in India, 2026
- IMARC: India personal finance software market
- Tricount: Splitwise alternatives in India
- ITVoice (News18): Splitwise free expense limit
- HyperVerge: Account Aggregator framework guide
- Sahamati: joining the Account Aggregator network
- Razorpay: NPCI guidelines and UPI intent
- Google Play: SMS and Call Log permissions policy
- Gemini API deprecations
- Google AI developer forum: Gemini 2.5 Flash availability
- Firebase AI Logic: supported models
- TaxGuru: DPDP Rules 2025 breakdown
- Consent.in: DPDP Rules
- AMSS regulatory alert: DPDP enforcement
- Kashif-E: AndroidLiquidGlass (API level table)
- PrismaIAGSL releases (three-tier fallback)
- Haze docs index (frame-time cost)
- Retirewise: budgeting apps in India, 2026
- Finny: best personal finance apps, 2026
- Pocket: Finance Tracker listing
- NETO listing (widgets)
- Firestore: transactions and batched writes (transactions fail offline)
- Firestore: write-time aggregations (count, sum and average are not cached)

Search results were read as snippets, not full pages, and some are secondary or forum sources. Verify dates and limits against the primary source before relying on them. Researched 1 November 2026.

18. Owner-approved decisions (v2.2)

These decisions come from the product owner and supplement sections 1 to 17. Nothing in sections 1 to 17 is removed. Where v2.2 conflicts with an earlier section, v2.2 wins and the conflict is listed below. Do not ask the owner to reopen a LOCKED decision.

Locked decisions

Area	Decision
Product	Fluxora: premium Indian household money management for students, roommates and young professionals
Core value	Track, budget, split and settle household finances
Look	Premium futuristic liquid glass (section 14)
Default theme and mode	Midnight Emerald, dark-first, with full light-mode support
Navigation	Floating glass pill
First release	Core finance plus every Wave 1 feature in section 16
Backend	A completely fresh Firebase project; never reuse old identifiers or credentials
Languages	English and Hindi
Money	Free at first; consider paid features after real usage
Identity	`applicationId` = `com.aistudio.financetrack.qwrxnv`; `namespace` = `com.example`. Change either only with a documented technical reason and owner approval

Area	Decision
Architecture	Kotlin, Compose, Material 3, MVVM + Repository, Gradle feature modules, Firestore, Firebase Auth, Cloud Functions, Firebase AI Logic, Remote Config, DataStore, WorkManager
Glass	A `:core:glass` module with the twelve components in section 14, one `GlassTokens` interface, three tiers (FULL, SOFT, FROST), and user settings Auto, Full, Reduced, Off. Quality changes never change layout or function
Glass library	Run the comparison of Haze, AndroidLiquidGlass or PrismalAGSL, and a minimal own AGSL version; choose one; hide it behind `GlassSurface`; no alpha dependency on essential screens without a working fallback

Conflicts and how they are resolved

#	Earlier text	v2.2 text	Resolution
1	Section 16 lists **eight** Wave 1 features	v2.2 says "all seven Wave 1 feature groups" but describes eight	All eight are in scope: smart quick-add, safe-to-spend, widgets, shortcuts and tile, subscription manager, rent and shared-bill templates, IOU ledger, weekly review. Owner to confirm the count
2	Receipt scan is Wave 2	Asks whether receipt scan belongs in Wave 1	Stays Wave 2 until the owner answers
3	Section 13 roadmap (security and glass early)	v2.2 priority order puts the financial core, offline and household integrity first, then Wave 1, then glass	v2.2 order wins; see the revised build order below. Design tokens, theme storage and the glass spike are small and may start early, but never block the core

#	Earlier text	v2.2 text	Resolution
4	Sections 5, 9 and 17 mention budget alerts and spending caps	v2.2: a budget alert is not a hard spending limit	Wording corrected below
5	Section 17 asks about the namespace	v2.2 locks it but also asks if it is permanent	Treat as locked for now; no change without approval
6	Section 16 says offline-first	v2.2 adds ten offline acceptance steps	Adopted; see offline rules below. This interacts with atomic bill payment (section 20, finding 1)

Revised build order

1. Basic financial core: money engine, auth, Firestore, transactions, categories, basic dashboard.
2. Verify transaction creation, persistence and dashboard figures with tests.
3. Offline behaviour, with the acceptance test below.
4. Household and recurring integrity: budgets, savings, recurring bills, roommate ledger, connection requests, settlements, phone ownership functions and their rules.
5. Wave 1 features.
6. Premium glass integration: bottom bar, Dynamic Island, Reactor, glass themes.
7. Performance work with measurements.
8. Security and privacy verification as the final gate. Security rules and functions are still written with each feature they protect, not left until here.
9. Release build.

Wave 2 and Wave 3 wait until step 9 passes.

Offline-first rules

Acceptance test, run on a device or emulator: open online; create a transaction; turn the network off; create another; navigate away and back; turn the network on; confirm both sync with no duplicate and none missing; confirm the dashboard matches the synced data.

- Never describe a write as server-confirmed while it is only pending. Use the Firestore snapshot metadata (pending writes, from cache) and show a small "saved on this phone" mark only on pending rows, in line with essential-first.
- Document the Firestore cache size and what offline queries can and cannot do.
- Operations that need the server (phone ownership, roommate acceptance, settlement confirmation, shared-expense creation) must show a clear "needs connection" state and never fake success.

Cost control (corrected wording)

A Firebase or Google Cloud budget alert **notifies** you; it does not stop spending. Hard limits come only from the Gemini Developer API spend cap, quotas, and limits we enforce in code. The plan must include: budget alerts; the Gemini spend cap; per-user daily AI request limits; function and Firestore quotas; usage monitoring; and `FLUXORA_COST_MODEL.md`. Keep all credentials outside source code. Until the owner finishes setup, mark each external integration AWAITING OWNER CONFIGURATION and keep developing.

Owner decision register (all open and locked items, merged)

#	Decision	Status	If unanswered
1	Legal contact emails for Ayush Mishra and Sunaina Bansal	OPEN, blocks release	Deployment config; no invented addresses
2	Support and grievance email	OPEN, blocks release	Same
3	Legal data fiduciary: individual or registered organisation	OPEN, blocks release	Same
4	Monthly Firebase and Google Cloud budget	OPEN, blocks production deploy	Develop on the emulator; no paid deploy until set
5	Extra Indian language after Hindi	OPEN	None; keep strings in resources
6	Is `com.example` permanent	OPEN, treated as locked	Keep
7	Receipt scan in Wave 1	OPEN	Wave 2

#	Decision	Status	If unanswered
8	Design references or Figma files	OPEN	Original design from section 14
9	Dashboard definitions in section 7 (balance, available to spend) and safe-to-spend rules in section 20	OPEN, owner approval needed	Accept as written
10	Target `minSdk` 26	OPEN	Lowest level the verified dependencies allow
11	Audio and receipt photos stay on device	OPEN	Yes
12	Tilt-driven highlight	OPEN	Off until the user opts in
13	Both flatmates confirm a settlement	OPEN (see the comment in section 2)	Both-party confirmation
14	Wave 1 in the first release; free first; Blaze plan	LOCKED	n/a

19. Execution directive and required deliverables

The coding agent must turn this specification into an implementation-ready blueprint, then start building. Do not only summarise or shorten the specification, and do not stop after planning. Keep chat replies short (essential-first, section 3); the blueprint lives in the documents listed here.

Step 1: inspect

Inspect the project and every supplied document first. Record what exists in `FLUXORA_IMPLEMENTATION_STATUS.md`.

Step 2: first deliverables (in this order)

1. **Specification audit** (A below).
2. **Owner decision register**, starting from section 18. Ask only essential questions, in one batch. Use documented defaults for everything non-blocking.

3. **Product architecture** and journeys (B below).
4. **Screen and navigation inventory** and the design system plan (C below).
5. **Database and security design** (E below).
6. **Financial test matrix** (D below).
7. **Dependency verification**: Kotlin, AGP, Gradle, KSP, Firebase BoM, Firebase AI Logic, Glance, chosen glass library, with the check date.
8. **Phase plan** (F below), then a **requirement traceability matrix** mapping every numbered requirement in sections 4 to 16 to a deliverable and a test.

Then begin implementation with the financial core.

A. Specification audit

Cover these twelve areas: contradictions; missing technical decisions; incomplete user journeys; unclear database ownership; financial edge cases; Firebase security gaps; offline sync risks; Android lifecycle problems; performance bottlenecks; dependency and API risks; privacy and legal gaps; missing acceptance tests. For each finding record: severity, affected section, technical explanation, proposed correction, and whether owner approval is needed. Do not silently change a requirement. Section 20 gives a starting list; the agent must extend it, not copy it.

B. User journeys

Design around complete journeys, not isolated screens: first-time onboarding; authentication; quick expense entry; income; budget creation; savings goals; recurring bills; roommate onboarding; shared expenses; UPI settlement; AI assistance; offline use; account deletion; data export. Every journey states: starting state, user action, expected UI response, backend operation, error handling, final state. One primary action per screen.

C. Design intelligence

Produce: design tokens, type scale, spacing system, component hierarchy, glass material hierarchy, the eight theme specifications with light and dark variants, navigation patterns, loading, empty and error states, and accessibility requirements. Then a screen-by-screen specification in `FLUXORA_DESIGN_MAP.md`. If reference designs are supplied, inspect each one and map its visual properties to reusable components; never say a supplied design is implemented when it is not. If none are supplied, create an original Fluxora design. Use only properly licensed assets. Add no unnecessary UI elements.

D. Financial verification document

A dedicated document of test cases for: integer paise arithmetic; Indian currency formatting; split allocation; rounding; negative balances; partial settlements; refunds; reversed contributions; recurring-bill retries; month-end dates; leap years; duplicate-transaction prevention. Every case lists input, expected output, calculation method and edge cases. The same vectors must run against both the Kotlin engine and the TypeScript functions so the two cannot drift.

E. Backend and security architecture

Produce the Firestore collection map, document schemas, security rules, Cloud Function contracts (typed input and output), the authentication flow, the phone ownership flow, a household authorization matrix, the settlement state machine, the offline sync strategy and the data deletion strategy. Mark every operation as CLIENT or TRUSTED (Cloud Function). Privileged financial operations are never trusted to the client.

F. Implementation plan

Split work into independently verifiable deliverables. Each lists: dependencies, files and modules, implementation steps, unit tests, integration tests, acceptance criteria and exit conditions. Put the financial core before visual effects. Wave 2 and Wave 3 never delay a verified core.

G. Evidence and documents

Every implementation claim needs evidence. Keep these current and honest, never describing work or verification that has not happened:

File	Holds
`FLUXORA_IMPLEMENTATION_STATUS.md`	Phases, features, results, blockers, next task
`FLUXORA_DESIGN_MAP.md`	Screens, components, tokens, states, design coverage
`FLUXORA_DATABASE_SCHEMA.md`	Collections, schemas, indexes, retention
`FLUXORA_SECURITY_AUDIT.md`	Threats, rules tests, function checks, findings
`FLUXORA_TEST_MATRIX.md`	Every test, what it proves, last result
`FLUXORA_RELEASE_CHECKLIST.md`	Release gates, store forms, legal identity

File	Holds
`FLUXORA_FIREBASE_SETUP.md`	The nineteen owner setup steps (project, app id, `google-services.json`, Google, email and phone sign-in, SHA fingerprints, Firestore, rules, indexes, Functions, AI Logic, Remote Config, App Check, Play Integrity, Crashlytics, Analytics, FCM)
`FLUXORA_COST_MODEL.md`	Expected cost, alerts, caps, per-user limits

Never mark an item complete until its acceptance criteria pass.

H. Final instruction

Implement, compile, test, correct and document one phase at a time. Do not dump all features in one response. The goal is not to look finished; it is to be finished, tested and buildable: a premium, secure, offline-capable, financially accurate native Android app.

20. Starting audit of v2.1 (gaps already found)

This is a head start for the agent's own audit (section 19, A). It is not complete. "Approval" means the owner must approve before the agent changes a stated requirement.

#	Severity	Section	Gap and why it matters	Proposed fix	Approval
1	High	7, 18	`markBillAsPaid` is specified as a Firestore transaction or a callable function, but Firestore transactions fail when the client is offline (Firestore docs), and callables need a network. This conflicts with offline-first	Offline path uses one batched write with deterministic ids: expense id and `billPayments` id built from bill and due date, and `nextDueDate` set to the computed next date. Rules check the three writes together (using `getAfter`). Online path may still use a function. Prove with an offline emulator test; batched writes are expected to queue locally	No (technical)

#	Severity	Section	Gap and why it matters	Proposed fix	Approval
2	High	7	Dashboard totals could use Firestore `sum()` or `count()`, but those aggregation queries do not support caching or real-time updates (Firestore aggregation docs), so they fail offline	Compute figures from cached documents for the selected period. Keep an all-time balance in monthly summary documents updated with `FieldValue.increment` in the same batch, plus a periodic reconcile function that corrects drift	No
3	High	7	Only INCOME and EXPENSE exist, so refunds and reversals have no model	Add a `kind` field: NORMAL, REFUND (links `refundOfId`, reduces that category's spend, not counted as income) and REVERSAL (audit-keeping correction). Deletes are soft with an audit trail	**Yes** (changes the two-type rule)
4	High	8	One settlement per balance; partial payments and overpayment are not modelled	A settlement carries its own `amountMinor` up to the outstanding amount; outstanding = shares minus confirmed settlements; overpayment is rejected or recorded as a credit the user must confirm	No
5	High	8	Firestore rules cannot reliably check that shares add up to the total or that remainders follow the fixed order	Shared expenses are created by a callable function using the same split algorithm as the app. Run shared golden test vectors in Kotlin and TypeScript. Creation needs a connection and says so	No
6	High	10	Deleting an account would remove shared expenses and settlements that other members rely on	Anonymise the member ("Former member"), keep amounts and shares, delete personal collections, release the phone record, and state the retention period in the privacy policy	**Yes** (legal and retention)
7	Medium	7	Safe-to-spend is under-defined (see rules below)	Adopt the definition below	**Yes** (financial definition)

#	Severity	Section	Gap and why it matters	Proposed fix	Approval
8	Medium	11, 14	The Reactor aims for the display refresh rate while full glass adds frame cost (Haze documents a measurable cost)	One performance budget per tier. Reduce glass first, then Reactor particles. Measure on mid-range and low-end profiles	No
9	Medium	10	Phone ownership via the ID token does not cover linking a phone to an existing email account, or a number already used by another Firebase user	Define linking with `linkWithCredential`, handle "credential already in use", and have the function reject unless the token carries the verified phone	No
10	Medium	16	Widgets and the weekly review run without live Firestore listeners and may show stale or sensitive data	A `WidgetSnapshotStore` written by the app on changes and refreshed by WorkManager, with an "as of" time, masked when masking or app lock is on, and empty when locked	No
11	Medium	7, 12	Day and month boundaries depend on device time zone, so travel could move a transaction between months	Store the local date as `yyyy-MM-dd` with a profile time zone (default Asia/Kolkata); derive month keys from it	No
12	Medium	12	CSV duplicate detection has no fingerprint, and two real purchases can look identical	Fingerprint = date + amount + category + normalised note. Show matches as "possible duplicate", skipped by default, user can override	No
13	Medium	6	Household access rules that fetch documents add reads and hit per-request limits on document lookups in rules	Keep a `memberUids` map on the household document and use one lookup per rule; check current rule limits	No
14	Medium	10	App Check and phone sign-in behave differently in debug, CI and unsigned builds	Document the dev flow: debug provider, test phone numbers, emulator use; production checks only on signed Play builds	No

#	Severity	Section	Gap and why it matters	Proposed fix	Approval
15	Low	16	"At least three similar transactions" is undefined, and a yearly subscription needs three years of history	Suggest amount within 5%, gap tolerance 3 days (weekly), 5 days (monthly), 15 days (yearly); for yearly accept two matches plus a user hint. Detection only suggests	**Yes** (thresholds)

Safe-to-spend today: exact rules (for owner approval)

1. `remainingBudget` = sum over categories that have a budget this month of `max(0, budget − spent)`, with refunds reducing spent.
2. `upcomingBills` = sum of unpaid bills whose next due date is on or before the last day of this month, including overdue ones. Paid bills are in `spent`, never in `upcomingBills`, so nothing is counted twice.
3. `daysLeft` = calendar days from today to month end, inclusive, in the profile time zone; always at least 1.
4. `safeToSpendToday = max(0, remainingBudget − upcomingBills) ÷ daysLeft`, integer division rounding down in paise.
5. If no budget exists this month the result is none and the UI shows a dash.
6. Open point for the owner: a bill in an unbudgeted category still reduces the figure, because it is paid from the same money. Confirm this is wanted.

Worked example: budgets Food ₹6,000 and Transport ₹2,000; spent Food ₹1,500 and Transport ₹2,500; unpaid bills ₹1,500; 10 days left. Food remaining is ₹4,500, Transport remaining is ₹0 (overspend does not go negative). (4,500 − 1,500) ÷ 10 = ₹300, which is 30,000 paise.

More acceptance tests to add

Offline create then reconnect without duplicates; offline bill payment then reconnect without a second expense; deletion of a household member while others keep their ledger; partial settlement then confirmation; widget shows nothing while locked; refund reduces category spend and not income; month boundary across a time-zone change; a theme and glass tier combination does not change layout.